

GEEKY

The Intellectual Self-Learning Platform

iOS • Android • Web

TECHNICAL REQUIREMENTS DOCUMENT (TRD)

Version 1.0 | Engineering Confidential

June 2026

Document Information

Field	Value
Product	Geeky — Intellectual Self-Learning Platform
Document Type	Technical Requirements Document (TRD)
Version	1.0
Date	June 2026
Status	Draft — Engineering Review
Platforms	iOS 16+, Android 10+, Web (PWA + SSR)
Backend Architecture	Microservices on AWS / GCP (hybrid)
Primary Language	TypeScript (full-stack)
AI Provider	Anthropic Claude API (claude-sonnet-4-6)
Prepared For	Engineering Lead, CTO, and Founding Engineers

1. System Architecture Overview

1.1 Architecture Pattern

Geeky follows a microservices architecture with an API Gateway layer, deployed on AWS (primary) with GCP Vertex AI for ML workloads. The frontend is a universal TypeScript monorepo — React Native for mobile, Next.js for web — sharing a common design system and API client layer.

Layer	Technology	Purpose
Mobile Client	React Native (Expo SDK 51)	iOS + Android native apps
Web Client	Next.js 14 (App Router) + PWA	Browser + offline support
Shared Design System	React Native + Tamagui / Nativewind	Cross-platform UI components
API Gateway	AWS API Gateway + Kong	Rate limiting, auth, routing
Auth Service	Node.js (NestJS) + JWT + OAuth2	Login, sessions, OTP
Content Service	Node.js (NestJS)	Subjects, topics, media management
AI Engine Service	Python (FastAPI)	Flashcard gen, quiz gen, debate AI
Debate Service	Node.js (NestJS) + WebSockets	Real-time threads, voting
Gamification Service	Node.js (NestJS)	XP, streaks, ranks, achievements

Layer	Technology	Purpose
Notification Service	Node.js + Firebase FCM + OneSignal	Push + in-app notifications
Search Service	ElasticSearch 8.x	Full-text content + user search
CDN / Media	AWS CloudFront + S3	PDF, image, thumbnail delivery
Database — Primary	PostgreSQL 16 (via AWS RDS Aurora)	Users, content, relations
Database — Cache	Redis 7 (AWS ElastiCache)	Sessions, leaderboards, rate limits
Database — Events	Apache Kafka	Activity events, streak triggers
ML / AI	Anthropic API + AWS Bedrock fallback	Flashcard, quiz, debate suggestions
Video	YouTube Data API v3 + custom embed	In-app video playback
Monitoring	Datadog + Sentry + AWS CloudWatch	APM, error tracking, infra

1.2 High-Level Service Map

Request flow: Client → CloudFront CDN → API Gateway → Auth Middleware → Relevant Microservice → PostgreSQL / Redis → Response. AI calls are async: Content Service enqueues to a Kafka topic → AI Engine consumes → stores results → notifies client via WebSocket.

2. Frontend Architecture

2.1 Tech Stack

Concern	iOS/Android (React Native)	Web (Next.js)
Framework	React Native 0.75 + Expo SDK 51	Next.js 14 (App Router)
Language	TypeScript 5.x	TypeScript 5.x
State Management	Zustand + React Query (TanStack Query v5)	Same
Navigation	Expo Router v3 (file-based)	Next.js App Router
UI Components	Tamagui + custom Greek Design System	Tailwind CSS + shadcn/ui adapted
Forms	React Hook Form + Zod validation	Same
Animations	React Native Reanimated 3 + Moti	Framer Motion
Offline / Cache	MMKV + React Query persistence	Service Worker + IndexedDB
Push Notifications	Expo Notifications + FCM	Web Push API + OneSignal

Concern	iOS/Android (React Native)	Web (Next.js)
Video Player	react-native-youtube-iframe	YouTube IFrame API
PDF Viewer	react-native-pdf	react-pdf
Flashcard Flip	Reanimated flipCard (custom)	CSS 3D transform component
Charts/Graphs	Victory Native / Skia	Recharts / Chart.js
Testing	Jest + React Native Testing Library	Jest + Playwright E2E
Analytics	Segment (client SDK)	Segment (browser SDK)

2.2 Monorepo Structure

Single repo managed with Turborepo:

```
geeky/
├── apps/
│   ├── mobile/           # Expo React Native (iOS + Android)
│   ├── web/              # Next.js 14 (PWA)
│   └── admin/            # Internal CMS (Vite + React)
├── packages/
│   ├── ui/               # Shared Tamagui component library
│   ├── api-client/       # Generated OpenAPI client (orval)
│   ├── schemas/          # Zod schemas shared across apps
│   ├── config/           # ESLint, TS configs
│   └── utils/            # Shared utility functions
├── services/             # Backend microservices
└── infra/                # Terraform + Docker configs
```

3. Backend Microservices Specification

3.1 Auth Service

Responsibilities

- User registration (email, phone OTP, OAuth)
- Session management (JWT access token 15 min, refresh token 7 days)
- OTP generation and verification (Twilio Verify API)
- OAuth 2.0 flows: Google, Apple, Facebook
- Password reset via email (SendGrid)

Key Endpoints

Method	Endpoint	Description
POST	/auth/register	Create account (email/phone/username)
POST	/auth/login	Login, returns access + refresh tokens

Method	Endpoint	Description
POST	/auth/otp/send	Send OTP to phone number
POST	/auth/otp/verify	Verify OTP, auto-login
POST	/auth/refresh	Refresh access token
POST	/auth/logout	Revoke refresh token
GET	/auth/oauth/:provider	Initiate OAuth flow
POST	/auth/password/reset	Send reset email

Security Requirements

- Bcrypt (cost factor 12) for password hashing
- Rate limiting: 5 failed logins → 15-min lockout (Redis-backed)
- OTP: 6-digit, 5-minute TTL, max 3 attempts per code
- Refresh token rotation on every use (RT reuse detection)

3.2 Content Service

Responsibilities

- CRUD for subjects, sub-topics, and all content types
- YouTube video metadata storage and chapter marker management
- PDF storage pointers (S3 signed URLs)
- News/article RSS ingestion (via cron job — every 6 hours)
- User progress tracking (content viewed, % completed)

Data Models (simplified)

Entity	Key Fields
Subject	id, name, domain, description, prerequisites[], prerequisite_ids[], icon_url, created_at
SubTopic	id, subject_id, name, order_index, estimated_minutes, created_at
Video	id, subtopic_id, youtube_id, title, duration_sec, chapter_markers[], transcript_text
Paper	id, subtopic_id, title, authors[], s3_key, doi, ai_summary, key_terms[]
Book	id, subtopic_id, title, author, s3_key, license_type, ai_reading_guide
NewsArticle	id, subtopic_id, title, url, source, relevance_score, published_at
UserProgress	user_id, content_id, content_type, watched_pct, completed, last_accessed_at

3.3 AI Engine Service (Python FastAPI)

Responsibilities

- Flashcard generation from content (video transcripts, paper summaries, book text)
- Quiz question generation with distractors and explanations

- Debate topic suggestion based on user's recent learning
- AI answer grading for short-answer quiz questions
- Spaced repetition scheduling (SM-2 algorithm implementation)

Flashcard Generation Pipeline

Trigger: Content Service publishes ContentCompleted event to Kafka. AI Engine consumes:

```
1. Fetch content text from Content Service (transcript / summary / extract)
2. Chunk text to ≤ 8,000 tokens
3. Call Claude API with flashcard generation prompt
4. Parse structured JSON response (card_type, front, back, difficulty_hint)
5. Store cards in PostgreSQL flashcards table
6. Publish FlashcardsReady event → Notification Service
```

Claude API Prompt Contract

Parameter	Value
Model	claude-sonnet-4-6
Max Tokens	4096
System Prompt	You are an expert academic content distiller. Extract flashcards from the following educational content. Return only valid JSON array — no markdown, no prose.
Response Format	JSON: [{"type": "definition concept formula timeline comparison", "front": string, "back": string, "tags": string[]}]
Temperature	0.3 (factual fidelity over creativity)
Fallback	AWS Bedrock Claude Sonnet if primary API unavailable

Quiz Generation

Similar pipeline to flashcards. Additional parameters:

- question_type: MCQ | true_false | fill_blank | short_answer
- difficulty: easy | medium | hard (derived from user's recent quiz performance)
- Each MCQ: 4 options, 1 correct, explanation field required
- Short answer: model answer + rubric for AI grading

3.4 Debate Service

Architecture

- WebSocket server (Socket.io) for real-time debate updates (new arguments, votes)
- REST API for CRUD of debates, arguments, voting
- Redis Pub/Sub for broadcasting to connected clients in a debate room
- Kafka consumer for AI topic suggestion events

Key Endpoints

Method	Endpoint	Description
GET	/debates	List debates (sort: trending recent relevant)
POST	/debates	Create debate thread
GET	/debates/:id	Debate detail + arguments tree
POST	/debates/:id/arguments	Post argument/rebuttal
POST	/arguments/:id/vote	Cast support/challenge vote
GET	/debates/trending	Top debates by engagement velocity
WS	/debates/:id/live	WebSocket: real-time argument stream

Toxicity Moderation

- Perspective API called async on every argument POST
- If toxicity score > 0.85: auto-hidden, placed in review queue
- If toxicity score 0.70–0.85: posted but flagged for human review
- Moderation dashboard in admin app; reviewed within 2 hours SLA

3.5 Gamification Service

XP Calculation

Action	XP Awarded
Complete a video lesson	10 XP
Complete a research paper read	25 XP
Complete a quiz (score $\geq 60\%$)	20 XP base + (score% $\times$ 0.3) bonus
Perfect quiz (100%)	50 XP
Revise a flashcard deck	5 XP per 10 cards
Post a debate argument	8 XP
Receive 10 'Support' votes on argument	15 XP
Win a debate (community vote)	40 XP
Maintain daily streak	5 XP $\times$ streak_day (capped at 50 XP/day)
First completion of a subject	200 XP

Global Rank Calculation

- Rank = positional rank by total XP across all users
- Rank recalculated in real-time for top 10,000; batched (every 5 min) for all others
- Redis Sorted Set (ZADD) used for $O(\log N)$ rank operations
- Country-level leaderboard: separate Sorted Set per country code

Streak Engine

- Kafka consumer listens to UserActivity events

- Activity types that count: video completed, quiz completed, flashcard deck reviewed, debate argument posted
- Streak check cron: runs daily at 00:05 UTC per user timezone
- Streak shield: stored as shield_count field; consumed before breaking streak

3.6 Notification Service

Channel	Technology	Use Case
Mobile Push	Expo Notifications + FCM (Android) + APNs (iOS)	Streak alerts, new content, rank changes
Web Push	Web Push API + VAPID keys + OneSignal	Debate notifications, new subjects
In-App	WebSocket broadcast + polling fallback	Real-time achievement unlocks, debate replies
Email	SendGrid (transactional templates)	Password reset, weekly digest, account events
SMS	Twilio SMS	OTP only; no marketing SMS

4. Database Architecture

4.1 PostgreSQL Schema — Core Tables

Table	Key Columns	Notes
users	id UUID, username, email, phone, password_hash, tier, created_at, timezone	Partitioned by created_at
subjects	id UUID, name, domain, description, prerequisites JSONB, icon_url, is_published	Full-text index on name, description
subtopics	id UUID, subject_id, name, order_index, estimated_minutes	FK to subjects
videos	id UUID, subtopic_id, youtube_id, title, duration_sec, transcript TEXT, chapter_markers JSONB	GIN index on transcript
papers	id UUID, subtopic_id, title, s3_key, ai_summary TEXT, key_terms TEXT[]	GIN index on key_terms
books	id UUID, subtopic_id, title, author, s3_key, license_type, ai_guide TEXT	
flashcards	id UUID, subtopic_id, user_id NULL (template) or UUID, type, front TEXT, back TEXT, tags TEXT[]	user_id NULL = AI-generated template
user_flashcard_state	user_id, flashcard_id, ease_factor, interval_days, next_review_at, repetitions	SM-2 state per user per card

Table	Key Columns	Notes
quizzes	id UUID, subtopic_id, question TEXT, type, options JSONB, correct_answer, explanation TEXT, difficulty	
user_quiz_attempts	id UUID, user_id, quiz_id, answers JSONB, score_pct, completed_at	Partitioned by completed_at
debates	id UUID, creator_id, subject_id, title, motion TEXT, status, vote_tally JSONB, ends_at	
arguments	id UUID, debate_id, parent_id, user_id, body TEXT, support_count, challenge_count, toxicity_score, is_hidden	Recursive CTE for threads
user_xp_log	id UUID, user_id, action_type, xp_delta, created_at	Append-only event log
achievements	id UUID, user_id, badge_id, earned_at	FK to badge catalog
streaks	user_id PK, current_streak, longest_streak, last_activity_date, shield_count	1-to-1 with users

4.2 Redis Data Structures

Key Pattern	Type	Purpose	TTL
session:{user_id}	String (JWT)	Active session tracking	7 days
otp:{phone}	String	OTP verification	5 minutes
rate:{ip}:login	Counter + EXPIRE	Login rate limiting	15 minutes
leaderboard:global	Sorted Set (ZADD)	Global XP ranking	No TTL (updated live)
leaderboard:country:{cc}	Sorted Set	Country leaderboard	No TTL
leaderboard:weekly	Sorted Set	Weekly XP (reset Mondays)	7 days
debate:{id}:votes	Hash	Live vote tallies	No TTL
user:{id}:streak	Hash	Current streak data (cache)	24 hours
flashcard_due:{user_id}	Set	Cards due today	24 hours
content_progress:{user_id}:{content_id}	String	Video watch %	30 days

4.3 Kafka Topics

Topic	Producer	Consumer(s)	Payload
content.completed	Content Service	AI Engine, Gamification	{user_id, content_id, content_type, subtopic_id}
flashcards.ready	AI Engine	Notification Service	{user_id, subtopic_id, card_count}

Topic	Producer	Consumer(s)	Payload
quiz.submitted	Content Service	Gamification, Notification	{user_id, quiz_id, score_pct, subtopic_id}
user.activity	Multiple services	Gamification (streak check)	{user_id, activity_type, timestamp}
debate.created	Debate Service	Notification, AI Engine	{debate_id, subject_id, title}
subject.published	Content Service	Notification Service	{subject_id, name, domain}
xp.awarded	Gamification	Leaderboard updater	{user_id, xp_delta, new_total}

5. AI System Design

5.1 Spaced Repetition — SM-2 Algorithm

Each flashcard review updates the card's ease factor and interval using the SuperMemo SM-2 algorithm:

```
def sm2(ease_factor, interval, repetitions, quality):
    # quality: 0=Again, 3=Hard, 4=Good, 5=Easy
    if quality < 3:
        repetitions = 0; interval = 1
    elif repetitions == 0:
        interval = 1
    elif repetitions == 1:
        interval = 6
    else:
        interval = round(interval * ease_factor)
    ease_factor = max(1.3, ease_factor + 0.1 - (5-quality)*(0.08 + (5-quality)*0.02))
    return ease_factor, interval, repetitions + 1
```

5.2 Adaptive Quiz Difficulty

User Score Range	Next Quiz Action
< 50% on last 3 quizzes	Serve 'easy' questions; re-serve incorrectly-answered cards
50–79%	Standard difficulty mix (60% medium, 30% easy, 10% hard)
≥ 80% on last 3 quizzes	Introduce 'hard' questions; reduce easy questions
100% on last 5 quizzes	Cross-topic challenge quiz (pulls from prerequisite subjects)

5.3 Debate AI Suggestions

- Event: user completes subtopic → AI Engine generates 2 debate topic suggestions
- Topics stored in debate_suggestions table with user_id and subtopic_id
- Notification Service sends in-app notification linking to Debate Arena
- Suggestion prompt template: 'Based on the following academic content, generate 2 debate motions suitable for structured intellectual discourse. Return JSON array [{"motion": string, "subject_tag": string}]'

6. Security Requirements

Concern	Requirement
Authentication	JWT RS256 signed tokens; HTTPS-only; HSTS headers
Authorisation	RBAC: roles = user moderator editor admin; enforced at service layer
Data Encryption	AES-256 at rest (AWS RDS + S3 SSE); TLS 1.3 in transit
PII	Phone numbers and emails encrypted in DB (pgcrypto); never logged
API Security	AWS WAF + rate limiting (100 req/min per user, 20/min unauth)
OWASP Top 10	Automated scans with OWASP ZAP in CI/CD pipeline
Dependency Scanning	Snyk integrated into GitHub Actions; block PRs with critical CVEs
Content Moderation	Perspective API for text; custom ML image classifier for avatars
Data Residency	User data for India stored in AWS ap-south-1 (Mumbai)
GDPR / DPDPA	Data deletion API; consent logging; DPO appointed; privacy manifest (iOS)
Penetration Testing	Annual external pentest; bug bounty programme (HackerOne)
Secret Management	AWS Secrets Manager; no secrets in code or env files in repo

7. Performance & Scalability

7.1 SLAs

Metric	Target
API P95 Response Time	< 200ms (read); < 500ms (write)
AI Flashcard Generation	< 8 seconds end-to-end (async OK; WebSocket push on completion)
WebSocket Debate Latency	< 150ms message delivery
Mobile App Load (cold start)	< 3 seconds on mid-range Android device
Web LCP (Largest Contentful Paint)	< 2.5 seconds on 4G

Metric	Target
PDF Load (S3 signed URL)	< 1.5 seconds (CloudFront edge cache)
Uptime SLA	99.9% monthly ($\leq$ 44 min downtime/month)
Database Query P99	< 50ms (indexed queries); < 500ms (analytical)

7.2 Scaling Strategy

- Horizontal pod autoscaling (HPA) in Kubernetes (EKS) for all services
- Aurora PostgreSQL auto-scaling read replicas (up to 5 at peak)
- Redis ElastiCache cluster mode — shard leaderboard sorted sets
- CDN caching: PDFs and video thumbnails at CloudFront edge (TTL 7 days)
- Kafka partition count: 12 partitions per topic; 3 broker cluster minimum
- AI Engine: GPU inference optional (CPU sufficient for Claude API calls — async)

8. Mobile — Platform Requirements

8.1 iOS

Requirement	Specification
Minimum iOS Version	iOS 16.0
Device Support	iPhone 12 and newer; iPad (secondary priority)
App Store	Category: Education; requires Apple Privacy Manifest + App Tracking Transparency
Push Notifications	APNs via Expo Notifications; requires explicit permission request
Background Refresh	Streak check runs on BGTaskScheduler
Offline	Flashcards downloaded to MMKV; viewed papers cached to FileSystem
Dark Mode	Follows system AND custom in-app toggle; stored in user preferences
Accessibility	VoiceOver support for all interactive elements; Dynamic Type respected
Deep Linking	Universal Links (apple-app-site-association) for debate and subject links

8.2 Android

Requirement	Specification
Minimum Android Version	Android 10 (API 29)
Target SDK	Android 14 (API 34)
Distribution	Google Play Store (primary); Huawei AppGallery (Phase 2)
Push Notifications	FCM via Expo Notifications; notification channels configured

Requirement	Specification
Background Work	WorkManager for streak checks and flashcard sync
Offline	Same as iOS — MMKV + FileSystem cache
Deep Linking	Android App Links (assetlinks.json)
Permissions	CAMERA (avatar upload), NOTIFICATIONS, INTERNET — no unnecessary permissions

8.3 Web (Next.js PWA)

Requirement	Specification
Framework	Next.js 14 App Router with SSR for SEO pages (subject catalogue, debate index)
PWA	next-pwa; Service Worker with Workbox; install prompt on desktop Chrome/Edge
Rendering Strategy	SSG for marketing pages; SSR for subject/debate pages; CSR for auth-gated content
SEO	next/metadata, Open Graph, JSON-LD structured data for subjects and debates
Performance	Turbopack; image optimisation via next/image; dynamic imports for heavy components
Offline	Service Worker caches flashcard decks and recent subject pages
Browser Support	Chrome 108+, Safari 16+, Firefox 109+, Edge 108+

9. Third-Party Integrations

Service	Purpose	Notes
Anthropic Claude API	Flashcard gen, quiz gen, debate suggestions, short-answer grading	claude-sonnet-4-6; AWS Bedrock fallback
YouTube Data API v3	Video metadata, thumbnail, embed	Requires API key; quota: 10,000 units/day (scale via multiple keys)
Twilio Verify	SMS OTP	₹0.06/OTP; throttle to 3/phone/hour
SendGrid	Transactional emails (reset, weekly digest)	Dedicated IP for deliverability
Firebase FCM	Android push notifications	Free tier sufficient to 500K users
Apple APNs	iOS push notifications	Requires Apple Developer account
OneSignal	Web push + push analytics dashboard	Free tier to 10K web subscribers
Google Perspective API	Debate content toxicity scoring	Free 1 QPS; paid scale
Segment	Cross-platform analytics event tracking	Feeds into Amplitude for product analytics

Service	Purpose	Notes
AWS S3 + CloudFront	PDF and image storage + CDN	S3 Intelligent-Tiering for cost optimisation
Stripe	Subscription billing (Scholar, Polymath tiers)	Razorpay as India-primary payment gateway
Razorpay	India payments (UPI, cards, netbanking)	Mandatory for INR transactions in India
DigiLocker API	Optional academic credential verification (future)	Phase 4 roadmap — institution licensing
HuggingFace (optional)	Sentence embedding for debate relevance matching	Alternative to Claude for embedding-only tasks

10. CI/CD & DevOps

Concern	Tool / Approach
Source Control	GitHub (monorepo) — branch strategy: main, develop, feature/*
CI	GitHub Actions — lint, type-check, unit tests on every PR
CD — Mobile	Expo EAS Build + EAS Submit for TestFlight / Play Internal Testing
CD — Backend	GitHub Actions → Docker build → AWS ECR → Helm chart deploy to EKS
CD — Web	Vercel (Next.js) for preview URLs per PR; production on custom domain via Vercel
Container Registry	AWS ECR
Orchestration	AWS EKS (Kubernetes) — HPA enabled per service
Infrastructure as Code	Terraform — all AWS resources managed as code
Environments	dev, staging (full replica), production
Database Migrations	Prisma Migrate — applied in CI before deployment; never manual
Secrets	AWS Secrets Manager — fetched at runtime via AWS SDK
Monitoring	Datadog APM + logs + dashboards; PagerDuty for on-call alerts
Error Tracking	Sentry (frontend + backend)
Load Testing	k6 — load tests run weekly on staging; blocking on > 10% regression

11. Data Privacy & Compliance

- India DPDPA (Digital Personal Data Protection Act 2023): Consent management, data principal rights (access, delete, correct)

- GDPR (for EU users in Phase 2): lawful basis documented; DPA signed with all sub-processors
- COPPA: Minimum age 13 enforced at signup; under-18 restricted from Debate Arena voting
- Apple App Store Privacy Manifest: All data types declared; NSPrivacyAccessedAPITypes listed
- Google Play Data Safety: Accurate disclosure of data collected, shared, encrypted
- Data retention: User data deleted within 30 days of account deletion request
- Audit logging: All admin actions and data access logged to append-only S3 bucket

12. Phased Technical Rollout

Phase	Engineering Deliverables	Timeline
Phase 0 — Setup	Monorepo, CI/CD, Infra (Terraform), Auth Service, DB schema v1	Weeks 1–3
Phase 1 — Core MVP	Content Service, AI Engine (flashcard + quiz), Gamification (XP, streaks), mobile + web shells	Weeks 4–10
Phase 2 — Debate	Debate Service + WebSocket, Notification Service, leaderboards, admin CMS	Weeks 11–14
Phase 3 — AI Depth	SM-2 spaced repetition v2, adaptive quiz, debate suggestion AI, offline mode	Weeks 15–18
Phase 4 — Monetisation	Stripe + Razorpay, tier gates, Scholar/Polymath feature locks, subscription management	Weeks 19–22
Phase 5 — Scale	Multi-region Aurora, Kafka scale, search (ElasticSearch), institution API	Weeks 23–26

GEEKY — TRD v1.0 | Engineering Confidential
"Give me a place to stand and I will move the Earth." — Archimedes